DSA STUDY BLUEPRINT SERIES

39. N Queens Problem Backtracking

Placing N Queens safely on Chessboard

Section 06 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Place N queens such that no two queens attack each other.
- Validate row, column, and diagonal constraints.

Memory & Execution Blueprint

C++ Implementation Reference

Standard code layout and syntax

```
bool isSafe(vector& board, int r, int c, int n) {
    for (int i=0; i=0 && j>=0; i--, j--) if (board[i][j]=='Q') return false;
    for (int i=r, j=c; i>=0 && j
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(N!)
Space Complexity	$O(N^2)$ (Board state)

Key Practices & Gotchas

- **Important:** Prune branch decisions early to reduce evaluation times.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace backtracking cell navigations in a grid puzzle:

Cell visited	Direction Checked	Move Validation	Dead-end Backtrack step
(0, 0)	Down (D) → (1, 0)	Valid (Open path)	Proceed to cell (1, 0)
(1, 0)	Right (R) → (1, 1)	Invalid (Blocked wall)	Revert to cell (1, 0) and try Next check

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pruning Searches:** Check safety criteria (e.g. diagonal checks for N-Queens) to skip bad board configurations immediately.
- **Visited state maps:** Mark board grid tiles as visited inline to avoid loops, resetting them when returning.
- **Related LeetCode Problems:** #51 N-Queens, #37 Sudoku Solver, #79 Word Search.